

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Davi Gomes Pinto Coelho

Metodologia para Quebra de uma Aplicação Monolítica em Microserviços

São João del-Rei

2026

UNIVERSIDADE FEDERAL DE SÃO JOÃO DEL-REI

Davi Gomes Pinto Coelho

Metodologia para Quebra de uma Aplicação Monolítica em Microsserviços

Monografia apresentada como requisito da disciplina de Projeto Orientado em Computação I do Curso de Bacharelado em Ciência da Computação da UFSJ.

Orientador: Prof. Dr. Matheus Carvalho Viana

Universidade Federal de São João del-Rei – UFSJ

Bacharelado em Ciência da Computação

São João del-Rei

2026

Davi Gomes Pinto Coelho

Metodologia para Quebra de uma Aplicação Monolítica em Microsserviços

Monografia apresentada como requisito da disciplina de Projeto Orientado em Computação I do Curso de Bacharelado em Ciência da Computação da UFSJ.

São João del-Rei, 19 de agosto de 2026:

Prof. Dr. Matheus Carvalho Viana
Orientador

São João del-Rei
2026

*Este trabalho é dedicado às crianças adultas que,
quando pequenas, sonharam em se tornar cientistas.*

Agradecimentos

Os agradecimentos principais são direcionados à Gerald Weber, Miguel Frasson, Leslie H. Watter, Bruno Parente Lima, Flávio de Vasconcellos Corrêa, Otavio Real Salvador, Renato Machnievscz¹ e todos aqueles que contribuíram para que a produção de trabalhos acadêmicos conforme as normas ABNT com L^AT_EX fosse possível.

Agradecimentos especiais são direcionados ao Centro de Pesquisa em Arquitetura da Informação² da Universidade de Brasília (CPAI), ao grupo de usuários *latex-br*³ e aos novos voluntários do grupo *abnT_EX2*⁴ que contribuíram e que ainda contribuirão para a evolução do abnT_EX2.

¹ Os nomes dos integrantes do primeiro projeto abnT_EX foram extraídos de <<http://codigolivre.org.br/projects/abntex/>>

² <<http://www.cpai.unb.br/>>

³ <<http://groups.google.com/group/latex-br>>

⁴ <<http://groups.google.com/group/abntex2>> e <<http://abntex2.googlecode.com/>>

*“Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2)*

Resumo

A evolução dos sistemas de informação frequentemente conduz aplicações legadas construídas sob arquiteturas monolíticas a gargalos de acoplamento, alta complexidade de manutenção e limitações de escalabilidade. Diante deste cenário, a transição para uma arquitetura de microsserviços surge como uma alternativa viável para promover o desacoplamento e a evolução independente dos componentes de software. Contudo, a migração sem um planejamento estruturado introduz riscos operacionais significativos. Este trabalho apresenta uma metodologia estruturada para a decomposição de aplicações monolíticas em microsserviços, fundamentada nos princípios de Domain-Driven Design (DDD) para o mapeamento de Bounded Contexts, no padrão Strangler Fig (Figueira-Mata-Pau) para estrangulamento gradual do monólito e no padrão Database per Service para garantir o isolamento de dados por meio da persistência poliglota. Para a validação prática do roteiro proposto, foi desenvolvida uma Prova de Conceito (PoC) containerizada com Docker simulando um e-commerce, composta por um front-end/BFF em Next.js e três microsserviços independentes utilizando MongoDB (Catálogo), Redis (Carrinho de Compras) e PostgreSQL (Pedidos e Pagamentos). Os resultados demonstraram que o roteiro metodológico viabiliza a segregação efetiva dos domínios de negócio e dos bancos de dados, assegurando baixo acoplamento, alta coesão e escalabilidade granular por serviço.

Palavras-chaves: Microsserviços. Domain-Driven Design. Strangler Fig Pattern. Database per Service.

Abstract

The evolution of information systems often leads legacy applications built on monolithic architectures to coupling bottlenecks, high maintenance complexity, and scalability limitations. In this context, migrating to a microservices architecture emerges as a viable alternative to promote decoupling and the independent evolution of software components. However, migration without structured planning introduces significant operational risks. This work presents a structured methodology for decomposing monolithic applications into microservices, based on Domain-Driven Design (DDD) principles for mapping Bounded Contexts, the Strangler Fig Pattern for gradual monolith strangulation, and the Database per Service pattern to ensure data isolation through polyglot persistence. For the practical validation of the proposed roadmap, a Docker-containerized Proof of Concept (PoC) of an e-commerce platform was developed, consisting of a Next.js frontend/BFF and three independent microservices using MongoDB (Catalog), Redis (Shopping Cart), and PostgreSQL (Orders and Payments). The results demonstrated that the methodological roadmap enables the effective segregation of business domains and databases, ensuring low coupling, high cohesion, and granular scalability per service.

Key-words: Microservices. Domain-Driven Design. Strangler Fig Pattern. Database per Service. Polyglot Persistence.

Sumário

1	INTRODUÇÃO	9
2	FUNDAMENTAÇÃO TEÓRICA	10
2.1	Conceituação de Arquitetura Monolítica	10
2.2	Conceituação de Arquitetura de Microserviços	11
2.2.1	Características Fundamentais	11
2.2.2	Desafios da Arquitetura de Microserviços	12
3	TRABALHOS RELACIONADOS	14
4	METODOLOGIA	15
5	CONCLUSÃO	16
	REFERÊNCIAS	17

1 Introdução

No cenário contemporâneo da indústria de software, a capacidade de entregar funcionalidades de forma rápida, confiável e frequente consolidou-se como um diferencial competitivo crítico. O modelo tradicional de desenvolvimento, historicamente caracterizado por ciclos de lançamento longos e transferências manuais entre equipes de desenvolvimento e operações, mostra-se cada vez mais inadequado para atender à demanda atual por agilidade e inovação contínua. Esse cenário de pressão por velocidade frequentemente expõe gargalos operacionais, onde a tentativa de acelerar entregas sem a devida automação resulta em processos manuais propensos a erros humanos e instabilidade em produção, gerando um ambiente de tensão e risco elevado a cada novo lançamento.

Em resposta a esses desafios operacionais, a indústria tem adotado abordagens que visam unificar o desenvolvimento e as operações através de uma cultura de colaboração e, fundamentalmente, através da automação de processos. No coração técnico dessa transformação estão as práticas de Integração Contínua e Entrega Contínua, orquestradas por pipelines de implantação automatizados. Embora os benefícios teóricos dessas práticas, como a redução de riscos e o aumento da qualidade, sejam amplamente reconhecidos, muitas organizações ainda encontram dificuldades para quantificar objetivamente o impacto real dessa transição tecnológica. Apenas recentemente o mercado passou a adotar métricas mais científicas de desempenho para desafiar a antiga crença de que a velocidade de entrega deve ocorrer necessariamente em detrimento da estabilidade do sistema.

Este Trabalho de Conclusão de Curso tem como objetivo analisar quantitativamente o impacto da implementação de um pipeline de CI/CD na eficiência da entrega de software. Através de um estudo experimental comparativo, utilizando aplicações web modernas em diferentes pilhas tecnológicas e um ambiente de nuvem emulado localmente para garantir a reprodutibilidade, este trabalho irá mensurar a diferença de desempenho entre um processo de deploy manual e um processo totalmente automatizado. A análise será fundamentada em métricas de tempo de entrega e frequência de implantação, buscando validar empiricamente como a automação atua como um catalisador para a alta performance na engenharia de software.

2 Fundamentação Teórica

A evolução dos sistemas de informação nas últimas décadas foi acompanhada por transformações significativas nos padrões de arquitetura de *software*. Com o crescimento da complexidade das aplicações e a exigência de entregas contínuas e escaláveis, os modelos tradicionais de desenvolvimento baseados em aplicações monolíticas passaram a apresentar limitações estruturais significativas. Nesse cenário, o estilo arquitetural de micro-serviços consolidou-se como uma abordagem proeminente para a construção de sistemas modernos, modulares e desacoplados.

2.1 Conceituação de Arquitetura Monolítica

Historicamente, a maioria dos sistemas de *software* foi desenvolvida sob o paradigma monolítico. De acordo com [Richardson \(2018\)](#), uma aplicação monolítica é concebida como uma unidade lógica singular de execução. Em um monólito típico, todos os componentes do sistema, tais como a interface com o usuário, a lógica de negócio e os módulos de acesso a dados, são empacotados e implantados conjuntamente em um único processo computacional.

Embora a arquitetura monolítica apresente vantagens nos estágios iniciais do ciclo de vida do *software*, como a simplicidade de testes locais, a facilidade de implantação inicial e a alta performance de chamadas de função diretas em memória, ela frequentemente enfrenta severos gargalos à medida que o sistema e a organização expandem-se, conforme cita ([NEWMAN, 2021](#)).

Um dos principais problemas decorre do alto acoplamento de código e de banco de dados. A proliferação de dependências diretas entre módulos e o compartilhamento de um único esquema de banco de dados relacional tornam o sistema rígido. Nessa estrutura, pequenas alterações em uma funcionalidade específica podem desencadear efeitos colaterais imprevistos em módulos aparentemente não relacionados ([RICHARDSON, 2018](#)).

Adicionalmente, a arquitetura monolítica impõe uma limitação severa à escalabilidade granular. Para escalar uma aplicação monolítica sob alta demanda, é necessário duplicar a unidade inteira de execução em múltiplos servidores. Esse processo consome recursos de infraestrutura de forma ineficiente, visto que módulos ociosos são replicados juntamente com aqueles que realmente necessitam de maior capacidade de processamento ([FOWLER; LEWIS, 2014](#)).

Por fim, manifesta-se o acoplamento no ciclo de implantação. Toda a aplicação precisa ser recompilada, testada integralmente e reimplantada a cada nova atualização, por

menor que seja a alteração no código-fonte. Esse fator eleva substancialmente a latência de entrega de novas funcionalidades e aumenta o risco operacional de falha global, onde uma exceção não tratada em um módulo secundário pode derrubar todo o processo da aplicação.

2.2 Conceituação de Arquitetura de Microserviços

Para suplantar as limitações estruturais do monólito, [Fowler e Lewis \(2014\)](#) definem a arquitetura de microserviços como uma abordagem em que a aplicação é projetada como um ecossistema de pequenos serviços autônomos. Cada serviço é executado em seu próprio processo computacional e se comunica com os demais por meio de mecanismos leves de rede, tipicamente utilizando APIs fundamentadas no protocolo HTTP com o padrão REST ou barramentos de mensageria assíncrona.

Segundo as diretrizes estabelecidas por [Newman \(2021\)](#), os microserviços são unidades de *software* independentemente implantáveis, fortemente coesas e estruturadas em torno de limites de negócios específicos. Em vez de organizar o código por camadas técnicas horizontais, tais como camada de apresentação, camada de regras de negócio e camada de persistência, os microserviços alinham a estrutura do *software* às capacidades de negócio da organização, promovendo o desacoplamento e a clareza de responsabilidade de cada componente.

2.2.1 Características Fundamentais

A literatura especializada destaca que a transição para microserviços fundamenta-se em um conjunto de características e princípios arquiteturais estruturantes. A primeira dessas características refere-se à componentização promovida por meio de serviços. Conforme enfatizado por [Fowler e Lewis \(2014\)](#), a componentização tradicional via bibliotecas ligadas em memória é substituída por serviços independentes expostos via rede. Essa mudança assegura que a substituição, atualização ou manutenção de um serviço específico ocorra sem a necessidade de recompilar ou reimplantar os demais componentes do ecossistema.

Em adição à substituição de bibliotecas por serviços de rede, sobressai-se a governança e a descentralização tecnológica. Diferente do monólito, que impõe uma única linguagem e um único *framework* para toda a aplicação, [Newman \(2021\)](#) ressalta que os microserviços viabilizam a adoção da abordagem da "ferramenta certa para o problema certo", ou programação poliglota, nas palavras do autor. Cada serviço possui autonomia para escolher a linguagem de programação, o *framework* e as bibliotecas que melhor se adaptem às suas exigências técnicas particulares ([NEWMAN, 2021](#)).

No que tange ao gerenciamento da informação, o isolamento de dados por serviço é apontado por [Richardson \(2018\)](#) como um pilar incontornável. Cada microserviço deve possuir e gerenciar seu próprio repositório de dados privado, sendo impedido o acesso direto de serviços externos ao seu banco de dados. Qualquer interação ou consulta a informações sob domínio de outro serviço deve ocorrer exclusivamente por meio de sua API pública. Esse princípio elimina o acoplamento oculto no nível de esquemas relacionais de banco de dados.

Dessa segregação de dados deriva diretamente a possibilidade de adoção da persistência poliglota. Como o banco de dados deixa de ser um recurso compartilhado globalmente, [Richardson \(2018\)](#) explica que é possível selecionar tecnologias de armazenamento distintas de acordo com a natureza de cada domínio de negócio. Por exemplo, um serviço de catálogo de produtos pode adotar um banco NoSQL orientado a documentos para otimização de leitura, enquanto o serviço de carrinho de compras utiliza um banco chave-valor em memória para alta performance, e o serviço de pedidos utiliza um banco relacional garantidor de transações ACID ([RICHARDSON, 2018](#)).

Por fim, a separação física entre processos contribui de maneira determinante para a resiliência operacional e o isolamento de falhas. Em uma arquitetura monolítica tradicional, uma exceção não tratada pode paralisar todo o processo computacional. Em contrapartida, sob o modelo de microserviços, a degradação ou a indisponibilidade pontual de um serviço específico não provoca o colapso de toda a aplicação. A adoção de padrões de resiliência impede a propagação de falhas em cascata, permitindo que funcionalidades não afetadas continuem operando normalmente para o usuário final ([NEWMAN, 2021](#)).

2.2.2 Desafios da Arquitetura de Microserviços

Apesar dos evidentes benefícios de disponibilidade e desacoplamento, a adoção de microserviços introduz uma nova camada de complexidade inerente aos sistemas distribuídos ([FOWLER; LEWIS, 2014](#)). Conforme adverte [Richardson \(2018\)](#), a principal contrapartida operacional reside na observabilidade e no diagnóstico de falhas. Enquanto em um monólito a depuração ocorre pela análise de um log único e unificado com a pilha de execução completa (*stack trace*), em microserviços a execução de uma única requisição de usuário atravessa múltiplos serviços independentes via rede.

Essa pulverização de processos torna a identificação das causas raízes de erros um desafio complexo. Para contornar essa dificuldade e correlacionar os registros de auditoria espalhados entre diversos *containers*, torna-se indispensável a implementação de técnicas de rastreamento distribuído (*distributed tracing*), injetando identificadores únicos de correlação em todas as chamadas de rede ([RICHARDSON, 2018](#)). Além disso, destacam-se como desafios a latência de rede adicional, a exigência de roteamento eficiente por meio de *API Gateways* e o gerenciamento de transações entre múltiplos repositórios.

Em particular, a substituição de transações ACID locais por modelos de consistência eventual exige a reformulação do fluxo de dados. A coordenação de estados entre serviços passa a demandar padrões avançados, tais como a Arquitetura Baseada em Eventos (*Event-Driven Architecture*) e o Padrão Saga ([RICHARDSON, 2018](#)). Dessa forma, a transição para microsserviços representa uma escolha consciente (*trade-off*): reduz-se a complexidade interna do código-fonte e ganha-se resiliência, em troca do aumento da complexidade de infraestrutura, rastreabilidade e comunicação de rede.

3 Trabalhos Relacionados

Nesta seção, são analisados estudos anteriores que investigaram abordagens de migração arquitetural, decomposição de aplicações monolíticas e os impactos da adoção de microsserviços na indústria de *software*.

4 Metodologia

Este capítulo apresenta a abordagem metodológica adotada para a concepção do roteiro de decomposição do monólito e a implementação da Prova de Conceito (PoC) de e-commerce.

5 Conclusão

Este capítulo apresentará as considerações finais, a avaliação dos resultados alcançados pela metodologia proposta e as direções para trabalhos futuros.

Referências

FOWLER, M.; LEWIS, J. *Microservices: a definition of this new architectural term*. 2014. Martinfowler.com. Disponível em: <<https://martinfowler.com/articles/microservices.html>>. Acesso em: 18.08.2026. Citado 3 vezes nas páginas 10, 11 e 12.

NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2nd. ed. Sebastopol, CA: O'Reilly Media, 2021. Citado 3 vezes nas páginas 10, 11 e 12.

RICHARDSON, C. *Microservices Patterns: With examples in Java*. Shelter Island, NY: Manning Publications, 2018. Citado 3 vezes nas páginas 10, 12 e 13.